A 124-Gb/s Decoder for Generalized Integrated Interleaved Codes

Wenjie Li, Jun Lin, *Senior Member, IEEE*, and Zhongfeng Wang, *Fellow, IEEE*

***Abstract*— Generalized integrated interleaved (GII) codes have attracted much attention in distributed storage systems since they can reduce the overall redundancy by creating redundancy shared by the interleaves. The Berlekamp-Massey (BM) algorithm makes the GII decoding efficient by reusing the preceding results at each decoding stage. However, it also forms the speed bottleneck of GII decoders. The reformulated inversionless Berlekamp-Massey (riBM) algorithm is well known for its short critical path, but it is not generally suitable for GII decoding since its polynomials are difficult to be reused directly. In this paper, we address this issue by reinitializing the polynomials and propose a riBM-based GII decoding algorithm. In our simulations, slight performance degradation induced by the first-layer miscorrection is observed in the high signal-to-noise ratio (SNR) region. Hence, a subtle method is introduced to handle the miscorrected interleaves. Furthermore, the hardware architecture of the proposed decoding algorithm is developed and a high throughput decoder for the given example GII code is implemented. The synthesis results under TSMC 28-nm technology show that the decoder can achieve a throughput of up to 124 Gb/s. Moreover, the decoder has been implemented into a field-programmable gate array (FPGA) to validate its error-correction performance.**

***Index Terms*— Generalized integrated interleaved (GII) codes, Reed-Solomon (RS) codes, hardware implementation, high-throughput decoder, reformulated inversionless Berlekamp-Massey (riBM) algorithm.**

I. Introduction

IN SOME digital systems, such as the storage systems and the optical communication systems, extremely low bit error rate (BER), frequently 1×10^{-15}, is required. In the region with such BER, as one of the most powerful error-correction codes, low-density parity-check (LDPC) codes suffer from severe error floor [1], [2], which is manifested as an abrupt change in the slope of error-rate curve. However, algebraic codes and their concatenated codes show advantages in achieving extremely high reliability because of their accurately analyzable performance. For the widely used algebraic codes, such as Bose-Chaudhuri-Hocquenghem (BCH) and Reed-Solomon (RS) codes, it is guaranteed in theory that there is no error floor in the region with BER around 1×10^{-15} under their hard decoding algorithms. However, the decoders of long RS codes and BCH codes in the application scenarios that require extremely low BER are computationally expensive. Algebraic concatenated codes composed of short component codes are more promising in low-complexity implementations.

Integrated interleaved (II) codes were originally proposed for storage systems [3]. They are two-level codes with redundancy shared by all interleaves. For the conventional RS codes, a trivial method to correct more errors is to increase the check symbols. Instead of increasing the check symbols of all interleaves, II codes create the shared redundancy to handle the extra errors. The nested scheme of II codes provides interleaves the ability to correct errors beyond their own error-correction capability. The generalizations of II codes, referred to as generalized integrated interleaved (GII) codes [4], provide more flexibility in code design and surpass II codes in error-correction performance. More nested codes with distinct error-correction capabilities are allowed to be constructed for GII codes. All interleaves belong to the first-level RS code and they are associated with nested codes by a specific linear relationship.

A conversion matrix can be defined to characterize the linear relationship between interleaves and nested codes. The original version [5] of conversion matrix is a submatrix of the parity-check matrix of RS codes. In [6], to improve the average locality, the original conversion matrix was simplified through directly replacing some entries by zeros. As the original and the simplified conversion matrices are both specific matrices, a generalized definition of the conversion matrix was given in [7]. On the generalized definition, a variety of matrices can be chosen as conversion matrices. With carefully selected matrices, the computations involved in the matrix multiplications can be reduced.

The systematic encoding algorithm [5], [8] for GII codes can be divided into two major parts. At first, the interleaves of the first level are encoded by the RS code with the smallest error-correction capability, which is also referred as the first-layer code. Then it proceeds to the iterative process, where one interleave is encoded in one iteration. In each iteration, a composite message vector is obtained first and an intermediate codeword is encoded using the corresponding nested code. Then the interleave is computed through the intermediate codeword and previous interleaves. Because of

Manuscript received December 28, 2018; revised March 28, 2019; accepted April 13, 2019. Date of publication April 26, 2019; date of current version July 3, 2019. This work was supported in part by the National Natural Science Foundation of China under Grant 61604068 and in part by the Fundamental Research Funds for the Central Universities under Grant 021014380065. This paper was recommended by Associate Editor G. Masera. *(Corresponding authors: Jun Lin; Zhongfeng Wang.)*

The authors are with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: jlin@nju.edu.cn; zfwang@nju.edu.cn).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCSI.2019.2911730

1549-8328 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

the similarities between GII and generalized error location (GEL) codes [9], [10] in coding scheme, they were compared and demonstrated equivalent [7], [11] in some cases.

The original hard decoding algorithm for GII codes was presented in [4], which is complicated because of handling the miscorrection of RS codes. With certain length and over certain field, the effect of miscorrection becomes weaker as the minimum distance of the RS code increases. For GII code whose first-layer RS code has reasonable large minimum distance, the performance degradation incurred by miscorrection is negligible. Accordingly, for such GII codes, some operations of the original decoding algorithm are cumbersome and redundant. An efficient GII decoding algorithm [5] is devised by ignoring the miscorrection of the first-layer code. Berlekamp-Massey (BM) algorithm is adopted in the decoding algorithm for the convenience of using the newly computed high-order syndromes.

For an RS code with error-correction capability t, the critical path of BM architecture [12] is two multipliers, $\lceil log_2(t+1)\rceil$ adders, one finite field inverter, and one multiplexer. To eliminate the inversion, inversionless Berlekamp-Massey (iBM) algorithm [12], [13] was proposed. The critical path of iBM architecture is reduced to two multipliers and $\lceil log_2(t+1)\rceil+1$ adders. Furthermore, reformulated inversionless Berlekamp-Massey (riBM) architecture [14] with critical path of only one multiplier and one adder provides more promises in ultrahigh-speed implementations. With the increasing speed demand of next-generation communications, such as optical communications over 400 Gb/s and beyond, GII decoders with shorter critical paths are more competitive. In this paper, an riBM-based decoding algorithm for GII codes is proposed. At each decoding stage, preceding results can be reused by reinitilizing the polynomials of key-equation solver (KES). Moreover, to make the top-level schedule more regular, the error-correction capability involved at each decoding stage is fixed.

Some works [15], [16] focusing on coding schemes of GII related codes can also be found. However, to the best of our knowledge, studies about the hardware architectures of GII decoders have not been reported in the literature. In this paper, we develop a high-speed architecture for the GII decoders based on the proposed decoding algorithm. A row-echelon conversion matrix whose diagonal elements are all identities is employed for our decoder. In addition, a subtle method is introduced to address the issue of miscorrection in the low-BER region. To make a good tradeoff between the throughput and the hardware complexity, partially and fully parallel architecture are adopted in the symbol level and interleave level, respectively. A high-rate GII code is constructed as an example code and its decoder is implemented. The synthesis results show that the decoder can achieve a throughput up to 124 Gb/s with clock frequency of 883 MHz. Furthermore, the error-correction performance of the code is demonstrated by implementing its decoder onto a field-programmable gate array (FPGA).

The rest of the paper is organized as follows. In Section II, GII codes are reviewed and then the proposed decoding algorithm is devised. In Section III, the decoding performance of GII codes is analyzed and a GII code is constructed as an example. Then the decoder architecture is developed in Section IV. Implementation results are given in Section V. Finally, Section VI concludes the paper.

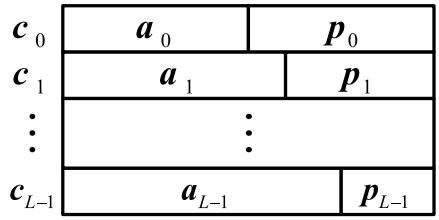


Fig. 1. An L-layer GII codeword, where $\boldsymbol{a}_l$ and $\boldsymbol{p}_l$ represent the message vector and parity vector, respectively.

II. Generalized Integrated Interleaved Codes

A. Definition of GII Codes

GII codes are multi-level codes over Galois field GF(2^q). They are composed of several RS codes, termed as interleaves, belonging to the first-layer code. Note that although GII codes can also be constructed by BCH codes [5], this paper focuses on GII codes over RS codes.

Let $C_{L-1}(n, k_{L-1}, t_{L-1})$, $C_{L-2}(n, k_{L-2}, t_{L-2}), \cdots, C_0(n, k_0, t_0)$ be L RS codes over GF(2^q), such that $C_0 \subseteq C_1 \subseteq \cdots \subseteq C_{L-2} \subseteq C_{L-1}$, where n, k_i, and t_i are the code length, code dimension, and error-correction capability, respectively. An L-layer GII code is defined as

$$C = \{\boldsymbol{c} = \left[\boldsymbol{c}_0, \cdots, \boldsymbol{c}_{L-1}\right], \boldsymbol{c}_i \in C_{L-1}, 0 \le i < L, \\ \hat{\boldsymbol{c}}_j = \sum_{i=0}^{L-1} \alpha^{ij} \boldsymbol{c}_i \in C_j, 0 \le j < \upsilon < L\}, \quad (1)$$

where α is a primitive element of GF(2^q). Nested codes $\hat{\boldsymbol{c}}_0, \cdots, \hat{\boldsymbol{c}}_{\upsilon-1}$ are more powerful codes than the first-layer code $C_{L-1}(n, k_{L-1}, t_{L-1})$, providing interleaves the ability to correct more than t_{L-1} errors. It is intuitive that more nested codes provide better error-correction performance. Without loss of generality, throughout this paper, GII codes with $\upsilon = L-1$ nested codes are considered.

A GII codeword can be arranged as a matrix over GF(2^q) as depicted in Fig. 1. The number of parity-check symbols of interleave $\boldsymbol{c}_i$ is equal to that of $C_i(n, k_i, t_i)$ for $i \le i < \upsilon$ despite that all interleaves belong to the first-layer code. In the systematic encoding process [5], parity vectors $\boldsymbol{p}_{L-1}, \cdots, \boldsymbol{p}_\upsilon$ are computed simultaneously at first. Then parity vectors $\boldsymbol{p}_{\upsilon-1}, \cdots, \boldsymbol{p}_0$ are computed iteratively and $\boldsymbol{p}_i$ only depends on $\boldsymbol{a}_i$ and the previous interleaves $\boldsymbol{c}_{L-1}, \cdots, \boldsymbol{c}_{i-1}$.

B. Conversion Matrix

The linear relationship between nested codes and interleaves as shown in (1) can be rewritten as

$$\left[\hat{\boldsymbol{c}}_0, \hat{\boldsymbol{c}}_1, \cdots, \hat{\boldsymbol{c}}_{\upsilon-1}\right]^T = \boldsymbol{H} \cdot \left[\boldsymbol{c}_0, \boldsymbol{c}_1, \cdots, \boldsymbol{c}_{L-1}\right]^T, \quad (2)$$

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

where

$$\boldsymbol{H} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & \alpha & \cdots & \alpha^{L-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{\upsilon-1} & \cdots & \alpha^{(\upsilon-1)(L-1)} \end{bmatrix} \tag{3}$$

is the conversion matrix. Obviously, $\boldsymbol{H}$ is a Vandermonde matrix. A simplified conversion matrix is introduced in [6] by directly replacing some entries by zeros. With such conversion matrix, the average locality is improved by accessing fewer interleaves when extra erasures need to be recovered.

The most important restriction on conversion matrix is that its submatrix composed of the first i rows and arbitrary i columns is invertible, for $0 < i \leq \upsilon$. The inverse constraint is required by solving the linear equations in the decoding algorithm, which will be discussed in the next subsection. It is easy to demonstrate that the original and the simplified matrices both satisfy the inverse constraint because of the characteristic of Vandermonde matrices.

In [7], the conversion matrix was studied in detail and its generalized definition was given. It is shown that any matrix subjecting to the inverse constraint determines a GII code. The generalized conversion matrix can be expressed as

$$\boldsymbol{H} = \boldsymbol{T} \cdot \boldsymbol{\Pi}. \tag{4}$$

$\boldsymbol{\Pi}$ is a row-echelon matrix, of which diagonal elements are all identity elements, satisfying that for $0 < i \leq \upsilon$, its submatrix composed of the first i rows and arbitrary i columns is invertible. $\boldsymbol{T}$ is a lower-triangular matrix with nonzero elements on the diagonal.

In this paper, to further simplify the computations involved in matrix multiplications, the conversion matrix of [6] is modified by multiplying its each row with a nonzero element. After the multiplications, the first nonzero element of each row is transformed into an identity element. Let $\boldsymbol{T}$ be an identity matrix, we have $\boldsymbol{H} = \boldsymbol{\Pi}$. The conversion matrix of the proposed 5-layer GII decoder is

$$\boldsymbol{H} = \boldsymbol{\Pi} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & \alpha & \alpha^2 & \alpha^3 \\ 0 & 0 & 1 & \alpha^2 & \alpha^4 \\ 0 & 0 & 0 & 1 & \alpha^3 \end{bmatrix}. \tag{5}$$

As mentioned before, the conversion matrix of [6] satisfies the inverse constraint. Considering any invertable submatrix consisting of the first i rows and arbitrary i columns, the elementary row operation, i.e. multiplying each row with a nonzero element, preserves the invertibility. Thus the above matrix $\boldsymbol{H}$ still satisfies the inverse constraint.

C. Conventional Decoding Algorithm

Taking the possible miscorrection of RS codes into account, the original hard decoding algorithm [4] is complicated. In fact, with certain length and over certain field, the effect of miscorrection becomes weaker as the error-correction capability of the RS code increases. For example, the error-correction capability of the first-layer code of our example code is 12 and no performance degradation is observed in the high-BER region (BER $> 1 \times 10^{-13}$) in our simulations. An efficient decoding algorithm ignoring the miscorrection is proposed in [5]. BM algorithm is adopted as the KES in this decoding algorithm and at each stage, the previous results can be reused. In this subsection, the conventional decoding algorithm of [5] is described.

For an RS code with error-correction capability t, denoting the received polynomial by $r(x)$, syndromes are computed as $S_i = r(\alpha^i)$, for $i = 1, 2, \cdots$. Assuming e errors with values $Y_1, Y_2, \cdots, Y_e$ have occurred during transmission, the error locations are denoted by $X_1 = \alpha^{i_1}, X_2 = \alpha^{i_2}, \cdots, X_e = \alpha^{i_e}$. Then error locator polynomial and error evaluator polynomial are defined as

$$\Lambda(x) = \prod_{i=1}^{e}(1 - X_i x) = 1 + \lambda_1 x + \cdots + \lambda_e x^e, \tag{6}$$

$$\begin{aligned} \Omega(x) &= \sum_{i=1}^{e} Y_i X_i^{m_0} \prod_{j=1, j\neq i}^{e} (1 - X_i x) \\ &= 1 + \omega_1 x + \cdots + \omega_{e-1} x^{e-1}, \end{aligned} \tag{7}$$

where m_0 is a constant and set to 1 in the proposed decoding algorithm. The relationship among the two polynomials and syndrome polynomial $S(x) = S_1 + S_2 x + \cdots + S_{2t} x^{2t-1}$ can be expressed as the key equation [17]: $\Lambda(x)S(x) = \Omega(x) \; mod \; x^{2t}$. KES solves the equation by iteratively updating $\Lambda(x)$ and $\Omega(x)$ from $S(x)$. In the iBM algorithms and its variants, such as riBM algorithm, error locator polynomial is the product of the originally defined error locator polynomial and a constant [12], implying that its first coefficient λ_0 may not be an identity element. Nevertheless, such polynomial and the originally defined error locator polynomial have the same roots and both of them can be used to find the error locations.

Hard decoding algorithms for GII codes are based on syndromes and contain multiple stages. Assuming some interleaves are determined as undecodable at one decoding stage, the syndromes of nested codes need to be computed and the high-order stage is required. At the next high-order stage, high-order syndromes of the undecodable interleaves are obtained through the syndrome conversion as shown in (10). Then the undecodable interleaves will be further decoded using the high-order syndromes. Similar to the decoding process of RS codes, the first three steps of each decoding stage are: 1) syndrome computation (SC), 2) KES, and 3) Chien search (CS). The fourth step is computing nested syndromes. CS is performed to find the error locations and then determine whether the interleave is decodable or not. An interleave is determined as decodable only when the number of found error locations is equal to the degree of $\Lambda(x)$. It should be noted that no nested syndromes need to be computed at the last stage. In the proposed algorithm, error evaluations for all interleaves are performed at the end of the last stage. Consequently, the fourth step of the last stage is error evaluation instead of nested syndrome computation. Forney algorithm is widely used in error evaluation and so does the proposed decoding algorithm. The decoding flow for an L-layer GII code is depicted in Fig. 2. Note that for different decoding algorithms, there may be some slight differences in their decoding flows.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

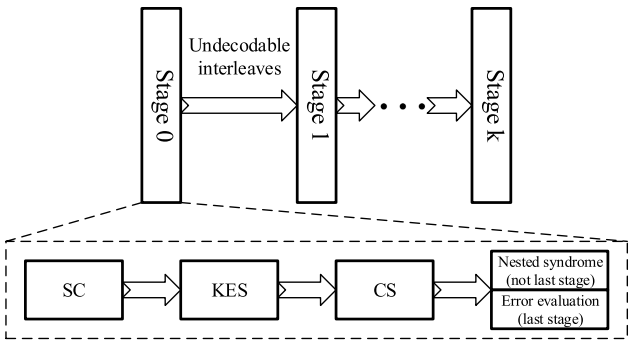


Fig. 2. The decoding flow for GII codes. In our proposed decoding algorithm, the last step of the last stage is error evaluation for all interleaves instead of computing nested syndromes. The number of stages depends on the specific decoding algorithm and there may exist some slight differences in the decoding flows for different decoding algorithms.

We now describe the conventional BM-based decoding algorithm devised in [5]. For the high-order stages, BM algorithm is slightly modified to be the BM updating algorithm. In such BM updating algorithm, the inputs include the syndromes, two polynomials $\Lambda(x)$ and $B(x)$, and two variables L_Λ and L_B. BM algorithm is well known and will not be presented in this paper. For the details of the slightly modified BM updating algorithm, please refer to the "Berlekamp-Massey Updating Algorithm" in [5]. Consider a transmitted RS codeword with $e \le t$ errors. After the $2t$ iterations of KES, the high-order syndromes can be computed by

$$S_i = \lambda_1 S_{i-1} + \lambda_2 S_{i-2} + \cdots + \lambda_e S_{i-e}, \quad i \ge 2t, \tag{8}$$

which is presented as *Lemma 1* in [5]. Instead of the commonly used Forney algorithm, in [5], the error values are evaluated by the Horiguchi-Koetter formula [17] [18]:

$$Y_i = \frac{X_i^{-(2t-2)}}{B(X_i^{-1})\Lambda'(X_i^{-1})}, \quad i = 1, 2, \cdots, e, \tag{9}$$

where $\Lambda'(x)$ is the formal derivative of $\Lambda(x)$.

The conventional GII decoding algorithm is presented in Algorithm 1. I^{un} and I^{new} represent the index sets of undecodable interleaves and newly decodable interleaves. The numbers of undecodable interleaves determined at the current stage and the previous stage are denoted by b and b', respectively. $\boldsymbol{H}_{I^{un}}^{-1}$ is the inverse of the submatrix consisting of the first b rows and the columns $I_0^{un}, \cdots, I_{b-1}^{un}$ of the conversion matrix $\boldsymbol{H}$. Because of the aforementioned inverse constraint and (2), the high-order syndromes can be computed through (10). At the first decoding stage, all interleaves are decoded using the first-layer code $C_{L-1}(n, k_{L-1}, t_{L-1})$. If there exist some undecodable interleaves, high-order stages will be required. The received interleaves are corrected as soon as they are determined as decodable. Obviously, the error-correction capability involved at each stage varies with the error patterns and so does the number of stages. Assuming there remain b undecodable interleaves after one decoding stage, t_{b-1} will be used to decode them at the next stage. All the required nested codes and their possibly required syndromes will be computed after the first stage if high-order stages are required. The computed nested syndromes should be stored during the decoding process and need to be updated through (11).

Algorithm 1 Conventional GII Decoding Algorithm

Input: received GII codeword $\boldsymbol{r} = \left[\boldsymbol{r}_0, \boldsymbol{r}_1, \cdots, \boldsymbol{r}_{L-1}\right]^T$.
1) Compute the syndromes $S_j^{(l)} = r_j(\alpha^l)$, for $0 \le j < L$, $1 \le l \le 2t_{L-1}$. Apply the BM algorithm to determine the error locator polynomials $\Lambda(x)$ for all interleaves. Use Chien search to find the roots of $\Lambda(x)$ and then determine the undecodable interleaves. Correct the received interleaves determined as decodable and then initialize b and I^{un}.
2) **if** $0 < b < L$ **then**
  2.1) Compute nested codes $\hat{\boldsymbol{r}}_j = \sum_{i \in I^{un}} h_{j,i}\boldsymbol{r}_i + \sum_{i \notin I^{un}} h_{j,i}\boldsymbol{r}_i'$, for $0 \le j < b$. Set $b' \leftarrow L$.
  2.2) Compute all high-order nested syndromes of the b nested codes $\hat{S}_j^{(l)} = \hat{r}_j(\alpha^l)$, $0 \le j < b$, $2t_{L-1} + 1 \le l \le 2t_j$.
  while $b > 0$ **do**
    2.3) Compute high-order syndromes:
    $$\left[S_{I_0^{un}}^{(l)}, \cdots, S_{I_{b-1}^{un}}^{(l)}\right]^T = \boldsymbol{H}_{I^{un}}^{-1} \cdot \left[\hat{S}_0^{(l)}, \cdots, \hat{S}_{b-1}^{(l)}\right]^T, \tag{10}$$
    for $2t_{b'-1} + 1 \le l \le 2t_{b-1}$.
    2.4) Apply the BM updating algorithm with the computed high-order syndromes to update the two polynomials $\Lambda(x)$ and $B(x)$ and the variables L_Λ and L_B for undecodable interleaves $I_0^{un}, \cdots, I_{b-1}^{un}$.
    2.5) Set $b' \leftarrow b$. Use Chien search to determine the undecodable interleaves and then update I^{un}, I^{new} and b. If $b' = b$, return the corrected interleaves and declare failure.
    2.6) Correct the decodable interleaves and compute high-order syndromes of decodable interleaves $S_j^{(l)}$ for $j \notin I^{un}$ by (8).
    2.7) Update the nested syndromes through
    $$\hat{S}_j^{(l)} \leftarrow \hat{S}_j^{(l)} - \sum_{i \in I^{new}} h_{j,i} S_i^{(l)}, \tag{11}$$
    for $0 \le j < b$, $2t_{b'-1} + 1 \le l \le 2t_j$.
3) **else**
  If $b = L$, return the received interleaves and declare failure. Otherwise, return the corrected GII codeword.

D. Proposed Decoding Algorithm

In the conventional decoding algorithm, if there remain b uncorrected interleaves after one decoding stage, the error-correction capability involved at the next stage will be t_{b-1}. Denoting the number of errors in one undecodable interleave by e, we have $e < t_i < t_{b-1}$ in some cases. Instead of t_{b-1}, error-correction capability t_i is sufficient to handle the errors. As larger error-correction capability implies larger number of KES iterations, there exist some redundant KES iterations that can be avoided.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON OF THE CONVENTIONAL AND THE MODIFIED SCHEDULES IN TERMS OF THE INVOLVED ERROR-CORRECTION CAPABILITIES FOR DIFFERENT STAGES

Schedule \ Stage	0	1	$\cdots$	$L-1$
Conventional	t_{L-1}	t_{b_0-1}	$\cdots$	$t_{b_{L-2}-1}$
Modified	t_{L-1}	t_{L-2}	$\cdots$	t_0

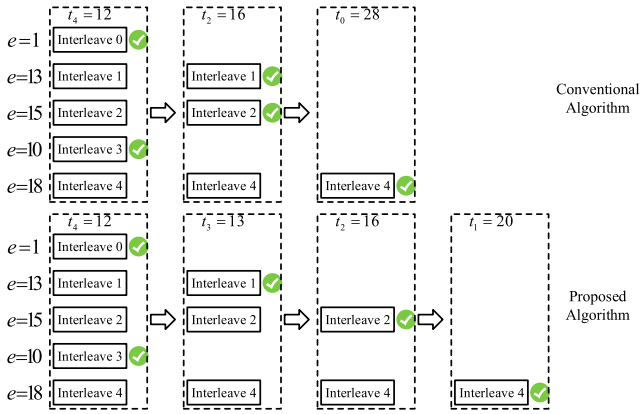


Fig. 3. The top-level decoding schedule comparison between the conventional and the proposed algorithms for the example code and an example error pattern.

To reduce the redundant operations and make the decoding schedule more regular, the schedule of the conventional decoding algorithm is modified. For an L-layer GII code, we fix the number of stages as L. The involved error-correction capabilities of the L stages are set to $t_{L-1}, \cdots, t_0$, respectively. Comparison between the conventional and the modified schedules in terms of the involved error-correction capabilities is listed in Table I. In this table, b_l is the number of undecodable interleaves after stage l. In the modified schedule, the closest larger error-correction capability t_i is employed when some interleaves fail to be decoded with t_{i+1}. Clearly, the modified schedule is independent of the distribution of errors. The regularity offers it some advantages in hardware designs. Consider a 5-layer GII example code with error-correction capabilities $(t_4, t_3, t_2, t_1, t_0) = (12, 13, 16, 20, 28)$, which will be constructed in Section III. Comparison between the conventional and the modified decoding schedules is shown in Fig. 3 by giving an example error pattern. The numbers of syndromes computed in the conventional and proposed algorithms are 168 and 146, respectively.

Note that the error pattern in Fig. 3 is just provided as an example to illustrate the differences between the conventional and the proposed algorithms. In this paper, we focus on the hardware implementations for the GII decodings. In addition to the above example error pattern, the worst cases should be also concerned. Assume the error pattern is $(12, 13, 16, 20, 28)$, i.e., the number of errors of each interleave is equivalent to the corresponding t_i, which is one of the worst cases. It is easy to see that, for such error pattern, the conventional and the proposed algorithms have the same decoding process, where the number of stages is 5. Thus their hardware architectures should be both designed with 5 stages.

In the steps 2.1) and 2.2) of conventional algorithm, all the possibly required nested syndromes are computed in advance. In fact, not all the nested syndromes need to be computed in the decoding process in most cases. To avoid the redundant computations and corresponding storages, we only compute nested syndromes when necessary. Still considering the example in Fig. 3, as shown in 2.2) of Algorithm 1, 56 nested syndromes are required and all of them are computed at the end of the first stage. However, in the proposed algorithm, the number of nested syndromes computed in the decoding is only 26. Moreover, different from correcting the received interleaves as soon as they are determined as decodable, all received interleaves are simultaneously corrected at the end of the last decoding stage in the proposed decoding algorithm. It implies that all interleaves remain unchanged before the error evaluations even if some of them are determined as decodable during the decoding process.

The critical path of the BM architecture employed in the conventional decoding algorithm is two multipliers, $\lceil log_2(t_0+1) \rceil$ adders, one finite field inverter, and one multiplexer. As the largest error-correction capability t_0 increases, such critical path becomes longer. The riBM algorithm is more prevalent because of its fairly short critical path of only one multiplier and one adder. To adopt riBM algorithm as the KES of GII decoding algorithm with the modified schedule, there are three issues to be addressed. The first one is how to update the polynomials at the high-order decoding stages. Using the BM updating algorithm, it is convenient to use preceding results and the newly computed high-order syndromes to update the polynomials. However, it is difficult to reuse the preceding results directly by exploiting riBM algorithm. The second one is the method to compute the high-order syndromes of decodable interleaves in riBM algorithm which is slightly different from that in BM algorithm. In addition, as all interleaves are corrected at the end of the last stage, only received interleaves are available to compute the nested syndromes during decoding.

In riBM algorithm, polynomial $\Delta(x, r)$ and $\Theta(x, r)$ are defined as $\Delta(x, r) = S(x)\Lambda(x, r)$, $\Theta(x, r) = S(x)B(x, r)$, where $r = 0, 1, \cdots$ is the iteration index and $B(x, r)$ is a "scratch" polynomial. $\hat{\Delta}(x, r)$ and $\hat{\Theta}(x, r)$ are two variants of $\Delta(x)$ and $\Theta(x)$. The i-th coefficients of $\hat{\Delta}(x, r)$ and $\hat{\Theta}(x, r)$ are the $(i+r)$-th coefficients of $\Delta(x, r)$ and $\Theta(x, r)$, i.e., $\hat{\delta}_i(r) = \delta_{i+r}(r)$ and $\hat{\theta}_i(r) = \theta_{i+r}(r)$ for $i = 0, 1, \cdots$. As a well-known algorithm, the details about riBM algorithm will not be presented in this paper.

To adopt riBM algorithm as the KES, the polynomials $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ of undecodable interleaves should be reinitialized before the KES iterations of the high-order stage. This is because the low-order coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ are variables of the newly computed high-order syndromes. Since the coefficients of $\hat{\Theta}(x)$ have the same updating regularity with $\hat{\Delta}(x)$, only $\hat{\Delta}(x)$ is discussed in the following contents. At the decoding stage with t_i $(0 < i < L)$, $S_1, \cdots, S_{2t_i}$ are available. After $2t_i$ KES iterations, the first $2t_i$ coefficients of $\hat{\Delta}(x)$

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

are

$$\begin{aligned}\hat{\delta}_0 &= \delta_{2t_i} = S_{2t_i}\lambda_1 + S_{2t_i-1}\lambda_2 + \cdots + S_1\lambda_{2t_i},\\ \hat{\delta}_1 &= \delta_{2t_i+1} = S_{2t_i}\lambda_2 + S_{2t_i-1}\lambda_3 + \cdots + S_2\lambda_{2t_i},\\ &\vdots\\ \hat{\delta}_{2t_i-1} &= \delta_{4t_i-1} = S_{2t_i}\lambda_{2t_i},\end{aligned} \tag{12}$$

where the iteration index is omitted for the sake of brevity. Note that the high-order coefficients are equal to zeros. Assuming the available syndromes are $S_1, \cdots, S_{2t_i}$, $S_{2t_i+1}, \cdots, S_{2t_{i-1}}$, after $2t_i$ iterations, the first $2t_{i-1}$ coefficients are

$$\begin{aligned}\hat{\delta}_0' &= \delta_{2t_i}' = \underline{S_{2t_i+1}\lambda_0} + S_{2t_i}\lambda_1 + S_{2t_i-1}\lambda_2 + \cdots + S_1\lambda_{2t_i},\\ \hat{\delta}_1' &= \delta_{2t_i+1}' = \underline{S_{2t_i+2}\lambda_0 + S_{2t_i+1}\lambda_1} + S_{2t_i}\lambda_2 + S_{2t_i-1}\lambda_3 +\\ &\quad \cdots + S_2\lambda_{2t_i},\\ &\vdots\\ \hat{\delta}_{2t_{i-1}-1}' &= \delta_{2t_{i-1}+2t_i-1}' = \underline{S_{2t_{i-1}}\lambda_{2t_i}}.\end{aligned} \tag{13}$$

In (12) and (13), the degree of $\Lambda(x)$ is no more than $2t_i$ because the number of iterations is $2t_i$. Although high-order syndromes are available in (13), the first $2t_i$ coefficients of $\Lambda(x)$ only depend on the first $2t_i$ low-order syndromes at the end of the $2t_i$-th iteration, which implies that the coefficients of $\Lambda(x)$ in (12) and (13) are equivalent. Compared with $\hat{\delta}_0$, $\hat{\delta}_1$, $\cdots$, $\hat{\delta}_{2t_i-1}$, the underlined items are introduced by the high-order syndromes $S_{2t_i+1}, \cdots, S_{2t_{i-1}}$. Thus, the coefficients of $\hat{\Delta}(x)$ for undecodable interleaves should be reinitialized by adding the underlined items before the $2(t_{i-1} - t_i)$ iterations of the next stage. On the other hand, the polynomials of decodable interleaves remain unchanged in the rest of the decoding stages.

When an interleave is determined as decodable, its error locator polynomial and syndrome polynomial are related by

$$S_j\lambda_0 + S_{j-1}\lambda_1 + \cdots + S_{j-e}\lambda_e = 0, \quad e < j, \tag{14}$$

which can be derived from Newton's identities [19]. Compared with (8), the most significant difference is the involved λ_0. Then we have

$$\begin{aligned}&S_{2t_i+1}\lambda_0 + \delta_{2t_i} = 0,\\ &S_{2t_i+2}\lambda_0 + S_{2t_i+1}\lambda_1 + \delta_{2t_i+1} = 0,\\ &\vdots\\ &S_{4t_i}\lambda_0 + S_{4t_i-1}\lambda_1 + \cdots + S_{2t_i+1}\lambda_{2t_i-1} + \delta_{4t_i-1} = 0,\\ &S_{4t_i+1}\lambda_0 + S_{4t_i}\lambda_1 + \cdots + S_{2t_i+2}\lambda_{2t_i-1} + S_{2t_i+1}\lambda_{2t_i} = 0,\\ &\vdots\end{aligned} \tag{15}$$

Consequently, the high-order syndromes can be computed as

$$\begin{aligned}S_{2t_i+1} &= \lambda_0^{-1}\delta_{2t_i},\\ S_{2t_i+2} &= \lambda_0^{-1}(S_{2t_i+1}\lambda_1 + \delta_{2t_i+1}),\\ &\vdots\\ S_{4t_i} &= \lambda_0^{-1}(S_{4t_i-1}\lambda_1 + \cdots + S_{2t_i+1}\lambda_{2t_i-1} + \delta_{4t_i-1}),\\ S_{4t_i+1} &= \lambda_0^{-1}(S_{4t_i}\lambda_1 + \cdots + S_{2t_i+2}\lambda_{2t_i-1} + S_{2t_i+1}\lambda_{2t_i}),\\ &\vdots\end{aligned} \tag{16}$$

As mentioned before, the interleaves are corrected at the end of the last decoding stage, which implies that only received interleaves are available before the error evaluations. We next show that how to compute nested syndromes through the received interleaves and the high-order syndromes of decodable interleaves. Denote the received polynomial and the error polynomial of interleave i by $r_i(x)$ and $e_i(x)$, respectively. Although nested syndromes are defined as the syndromes of nested codes, for the sake of convenience, henceforth we redefine nested syndrome $\hat{S}_j^{(l)}$ of nested code j as the linear combination of the high-order syndromes of undecodable interleaves:

$$\hat{S}_j^{(l)} = \sum_{i \in I^{un}} h_{j,i} e_i(\alpha^l), \quad 0 \le j < \upsilon, \ 1 \le l \le 2t_j, \tag{17}$$

where $h_{j,i}$ is the entry of $\boldsymbol{H}$. Note that $\hat{\boldsymbol{c}}_j \in C_j$ and then

$$\hat{c}_j(\alpha^l) = \sum_{i=0}^{L-1} h_{j,i} c_i(\alpha^l) = 0, \ 0 \le j < \upsilon, \ 1 \le l \le 2t_j. \tag{18}$$

We have

$$\begin{aligned}\hat{S}_j^{(l)} &= [\sum_{i=0}^{L-1} h_{j,i} c_i(\alpha^l) + \sum_{i \notin I^{un}} h_{j,i} e_i(\alpha^l) + \sum_{i \in I^{un}} h_{j,i} e_i(\alpha^l)]\\ &\quad - \sum_{i \notin I^{un}} h_{j,i} e_i(\alpha^l)\\ &= \sum_{i=0}^{L-1} h_{j,i} r_i(\alpha^l) - \sum_{i \notin I^{un}} h_{j,i} S_i^{(l)},\end{aligned} \tag{19}$$

for $0 \le j < \upsilon$, $1 \le l \le 2t_j$. $S_i^{(l)}$ is the high-order syndrome of decodable interleave i computed by (16).

With the modified schedule and the three devised methods, the proposed GII decoding algorithm is summarized in Algorithm 2. The error values are computed by the formulated Forney algorithm:

$$Y_i = \frac{X_i^{-2t}\Omega^{(h)}(X_i^{-1})}{\Lambda'(X_i^{-1})}, \quad i = 1, \cdots, e, \tag{20}$$

where $\Omega^{(h)}(x)$ is a degree-$(t-1)$ polynomial whose coefficients are equal to the first t coefficients of $\hat{\Delta}(x)$.

In Section IV, after developing the hardware architecture of the proposed decoding algorithm, we will show that the critical path of KES forms the speed bottleneck of GII decoders. Thus the proposed riBM-based decoding algorithm provides more promises in ultrahigh-speed implementations than the conventional BM-based algorithm.

III. Code Design

Under a memoryless channel, let p_b denote the bit error probability and then symbol error probability is computed as

$$p_s = \sum_{i=1}^{r} \binom{r}{i} (p_b)^i (1 - p_b)^{r-i}, \tag{21}$$

where r is the number of bits that a symbol consists of. Let

$$\phi_\omega = \binom{n}{\omega} (p_s)^\omega (1 - p_s)^{n-\omega} \tag{22}$$

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Proposed GII Decoding Algorithm

Input: received GII code $\boldsymbol{r} = \left[\boldsymbol{r}_0, \boldsymbol{r}_1, \cdots, \boldsymbol{r}_{L-1}\right]^T$.
for $i = L-1, L-2, \cdots, 0$ **do**
  Step 1: SC
  if $i = L-1$ **then**
    Compute low-order syndromes for all interleaves: $S_j^{(l)} = r_j(\alpha^l)$, for $0 \le j < L$, $1 \le l \le 2t_i$.
  else
    Compute high-order syndromes for undecodable interleaves: $\left[S_{I_0^{un}}^{(l)}, \cdots, S_{I_{b-1}^{un}}^{(l)}\right]^T = \boldsymbol{H}_{I^{un}}^{-1} \cdot \left[\hat{S}_0^{(l)}, \cdots, \hat{S}_{b-1}^{(l)}\right]^T$, for $2t_{i+1}+1 \le l \le 2t_i$.
  Step 2: KES
  if $i = L-1$ **then**
    Initialize polynomials in the same way with riBM algorithm with t_{L-1}.
  else
    Reinitialize the polynomials $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ of undecodable interleaves by (13).
  Perform $2(t_i - t_{i+1})$ iterations ($t_L = 0$) for undecodable interleaves.
  Step 3: CS
  Use Chien search to determine the decodable interleaves and update I^{un} and b. If $b > i$, declare decoding failure and set $i \leftarrow 0$.
  Step 4: Nested syndromes or error evaluations
  if $i = 0$ **then**
    Compute the error values and correct the decodable GII codeword.
  else
    Compute the high-order syndromes $S_{j'}^{(l)}$ of decodable interleaves and high-order nested syndromes $\hat{S}_j^{(l)}$ by (16) and (19), respectively, for $j' \notin I^{un}$, $0 \le j < b$, $2t_i + 1 \le l \le 2t_{i-1}$.

be the probability that an interleave has ω erroneous symbols. Decoding of GII codes fails in the following mutually exclusive situations:

1) For $i = v, v-1, \cdots, 1$, at least $i+1$ interleaves have e errors while the remaining interleaves have e' errors such that $t_i < e \le t_{i-1}$ and $e' \le t_i$.

2) At least one interleave has e errors while the remaining interleaves have e' errors such that $t_0 < e$ and $e' \le t_0$.

The decoding failure probability is

$$P_{GII} = \sum_{b=1}^{L} \binom{L}{b} \Big(\sum_{\omega=t_0+1}^{n} \phi_\omega\Big)^b \Big(\sum_{\omega=0}^{t_0} \phi_\omega\Big)^{L-b} + \sum_{i=1}^{v} \sum_{b=i+1}^{L} \binom{L}{b} \Big(\sum_{\omega=t_i+1}^{t_{i-1}} \phi_\omega\Big)^b \Big(\sum_{\omega=0}^{t_i} \phi_\omega\Big)^{L-b}. \quad (23)$$

Note that this equation only differs from that of [5] in terms of notations. All the discussion is under the assumption that with reasonable large error-correction capability, the performance degradation induced by miscorrection is negligible.

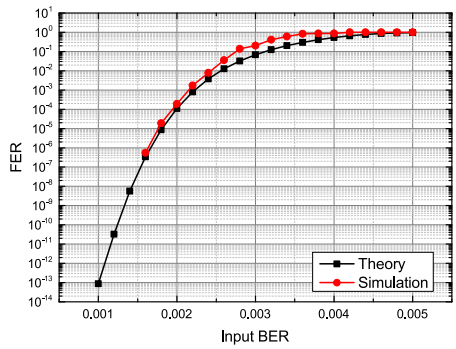


Fig. 4. Simulation results of the constructed (22995,21393) GII code in BSC. The curve labeled as "Theory" is drawn from (23) and another curve is drawn from the simulation results.

We now construct a 5-layer GII code over GF(2^9) as an example code. It should be noted that such code is just given as an example to demonstrate the validation of (23) and make it convenient to show the details of the decoder architecture. The length of interleaves is set to be the primitive length of 511. The performance restriction is that output frame error rate (FER) achieves 1×10^{-13} at input BER of 1×10^{-3}. By exhaustive search, the code with the maximum code rate is chosen. The error-correction capabilities of the chosen code are $(t_4, t_3, t_2, t_1, t_0) = (12, 13, 16, 20, 28)$. Simulation results of the constructed code under binary symmetric channel (BSC) are shown in Fig. 4.

The uncorrectable codewords in our simulations were observed and we found that all of them indeed possessed the error patterns that cannot be decoded correctly by the GII decoder. However, in the high-BER region, the simulation performance curve is inferior to that drawn from (23). We attribute the phenomenon to the inaccuracy of the computations of double-precision floating-point numbers in (23) and the nonideal noise generator made by the software program. It is shown that the two performance curves coincide better as the input BER declines. However, in our simulations performed on FPGA for the constructed code, when FER achieved about 1×10^{-10}, the miscorrection of the first-level RS code led to slight performance degradation. In the miscorrected cases, some undecodable interleaves are mistakenly determined as decodable and the errors are propagated to the higher-order decoding stages, leading to an unexpected decoding failure.

In the proposed decoder, a criterion is introduced to find the interleave suffering from the miscorrection of the first-layer RS code. As observed in our simulations, there is only one interleave with more than $t_4 = 12$ errors in the miscorrected cases. At the first decoding stage, all interleaves are determined as decodable while the interleave with errors more than 12 results in the miscorrection of its RS decoder. The method introduced to address the issue is: at the first decoding stage, if all interleaves are determined as decodable and one interleave is found with exact 12 errors, such interleave will be judged as suspicious and enforced to be decoded at the next stage. When there are several suspicious interleaves, only the first one will be further decoded. With such method, the performance degradation induced by miscorrection is eliminated. More details about the FPGA simulation results will be discussed in Section V.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

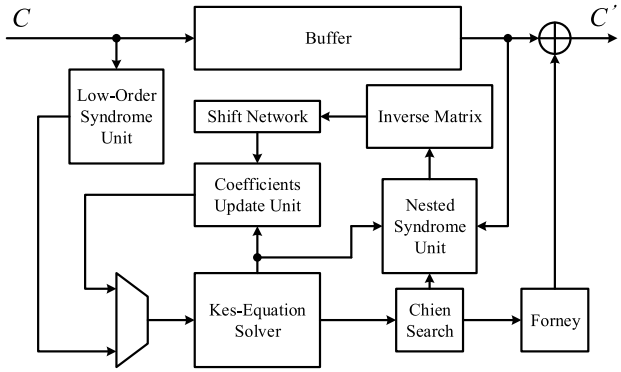


Fig. 5. Overall architecture of the proposed GII decoder.

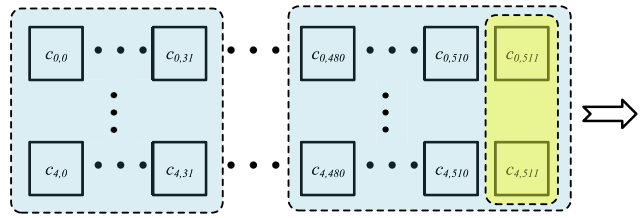


Fig. 6. The transmission format of the codeword. Every 32 columns of the codeword matrix are grouped to be transmitted in one clock cycle and totally 16 clock cycles are required by one codeword.

TABLE II
THE NUMBERS OF CLOCK CYCLES REQUIRED BY DIFFERENT OPERATIONS

Operation	Clock Cycles	Operation	Clock Cycles
LOSU	16	CS2(NSU2)	18
KES0(t=12)	24	IM+RP	2
CS0(NSU0)	19	KES3(t=20)	8
IM+RP	2	CS3(NSU3)	18
KES1(t=13)	2	IM+RP	2
CS1(NSU1)	18	KES4(t=28)	16
IM+RP	2	CS4(Forney 4)	16
KES2(t=16)	6	Total	169

IV. PROPOSED GII DECODER

To the best of our knowledge, no work about the hardware architectures of GII decoders has been reported in the literatures. This paper is the first to develop a decoder architecture for GII codes. In this section, the GII decoder architecture for the constructed example code is described in detail.

A. Overall Architecture

The target throughput of our design is over 100 Gb/s to meet the demand of high-speed communication systems such as the optical transport network (OTN). To achieve such high throughput, coarse- and fine-grained parallelism are both adopted in the architecture. The overall architecture is depicted in Fig. 5. Full parallelism is adopted in the interleave level and some modules such as SC and CS are partially parallel in symbol level. Low-order syndrome unit (LOSU) computes the low-order syndromes of all interleaves and at the same time the codewords are stored into the codeword buffer. At the first stage, low-order syndromes are used to initialize the polynomials of KES. At the high-order stages, the polynomials are reinitialized by the outputs of the coefficients update unit (CUU). CS is performed at the end of each stage to determine the decodable interleaves. Forney algorithm as shown in (20) is adopted to evaluate the error values. All error values are only computed at the end of the last stage and then subtracted from the codeword read from the buffer. While performing CS, nested syndrom unit (NSU) computes the nested syndromes through the polynomials of KES and the stored interleaves. To compute the high-order syndromes of undecodable interleaves from the nested syndromes, inverse matrix (IM) is used to perform a matrix inversion. Instead of a generalized inverter over the Galois field, it is implemented by an look-up table (LUT). A shift network is exploited to reorder the outputs of IM.

Transmission of codeword $\boldsymbol{C}$ is fully parallel in interleave level as shown in Fig. 6. For each clock cycle, 32 symbols of each interleave are transmitted and 16 clock cycles are required by one codeword. The symbols $c_{i,511}$ for $i = 0, \cdots, 4$ are the zero-pad symbols and will not be transmitted.

Fig. 7 shows the decoding flow of the proposed decoder. The number of clock cycles required to decode one codeword is totally 169. To avoid memory access conflicts, after the first 32 symbols of codeword $\boldsymbol{C}_i$ are read out, the new received symbols of $\boldsymbol{C}_{i+1}$ are stored into the buffer in the next clock cycle and its low-order syndromes are also computed at the same time. Two adjacent codewords are overlapped in 15 clock cycles. At the first stage, the polynomials of KES for all interleaves are initialized by the low-order syndromes. At each high-order stage, additional one clock cycle is needed to reinitialize the polynomials of undecodable interleaves. Error values of all interleaves are evaluated by Forney algorithm at the end of the last stage. If all interleaves are determined as decodable or a decoding failure is declared, the decoder will be frozen until the error value evaluation starts. Table II lists the numbers of clock cycles required by different operations, where RP denotes the reinitialization of polynomials. Except the last stage, where CS is performed at the same time with error evaluations under Forney algorithm, CS is performed while the nested syndromes are computed. It will be shown in Section V that our proposed decoder can achieve a clock frequency up to 833 MHz, using TSMC 28-nm CMOS technology.

B. Nested Syndrome Unit

If there are b undecodable interleaves, the NSU computes nested syndromes of order l by the following equation rewritten from (19):

$$\begin{bmatrix} \hat{S}_0^{(l)} \\ \hat{S}_1^{(l)} \\ \vdots \\ \hat{S}_{b-1}^{(l)} \end{bmatrix} = \boldsymbol{H}_{sub} \cdot \begin{bmatrix} r_0(\alpha^l) - k_0 \\ r_1(\alpha^l) - k_1 \\ \vdots \\ r_{L-1}(\alpha^l) - k_{L-1} \end{bmatrix}, \tag{24}$$

for $k_i = \begin{cases} S_i^{(l)}, & \text{if } i \notin I^{un}, \\ 0, & \text{else.} \end{cases}$ $\boldsymbol{H}_{sub}$ is the submatrix consisting of the first b rows of the conversion matrix.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

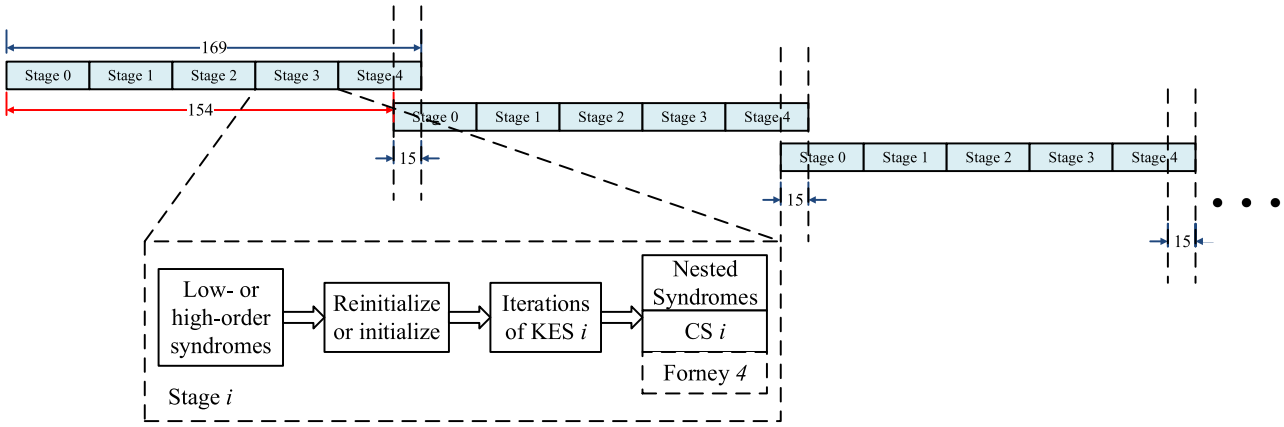


Fig. 7. The decoding flow of the proposed decoder. Every two adjacent codewords are overlapped in 15 clock cycles. CS and nested syndrome computation are performed simultaneously at the first four stages. At the last stage, nested syndromes are no longer required and CS and error evaluation are performed simultaneously.

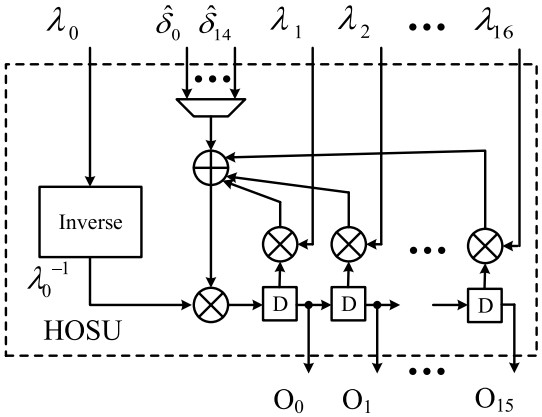


Fig. 8. The high-order syndrome unit for one interleave. High-order syndromes are stored in the shift registers in a decreasing order.

The computation is divided into three parts: the first one computes the high-order syndromes $S_i^{(l)}$ of decodable interleaves through (16); the second one computes $r_i(\alpha^l)$ and subtracts k_i from it; and the last one performs the matrix multiplications for different syndrome orders. Since the clock cycles required by CS are inevitable, we compute the nested syndromes at the same time with CS to avoid additional clock cycles.

Before the completion of CS, it is unknown about which interleave is decodable at current stage. Thus, besides the decodable interleaves determined at previous stage, the high-order syndromes of all the remaining interleaves are computed by NSU. After the determination of decodable interleaves, only high-order syndromes of decodable interleaves will be subtracted from $r_i(\alpha^l)$. Considering the example code, the numbers of high-order syndromes required by each stage for one interleave are $2\times(t_3-t_4) = 2$, $2\times(t_2-t_3) = 6$, $2\times(t_1-t_2) = 8$, and $2 \times (t_0 - t_1) = 16$, respectively. Note that no high-order syndromes need to be computed at the last stage. Since the maximum number of high-order syndromes is 16, which is close to the number of clock cycles required by CS, the high-order syndromes are designed to be computed serially.

The high-order syndrome unit (HOSU) for one interleave is shown in Fig. 8. Shift registers are used to store the high-order syndromes in a decreasing order from left to right. The values of registers are shifted to the right and the newly computed syndrome is stored in the first register. Although the fully serial architecture is adopted in the proposed decoder because of its lowest hardware complexity, fully parallel architecture can also be exploited to compute all the high-order syndromes simultaneously at the cost of much higher hardware complexity. It should be noted that HOSU in fully parallel is feed-forward and thus pipelining can be applied easily.

As mentioned before, high-order syndromes of all interleaves are computed even though some interleaves will be determined as undecodable after the CS. For the undecodable interleaves, the values computed by HOSUs are not their real high-order syndromes. Besides, those stored values will affect the computation of the higher-order syndromes at the next stage. Accordingly, before computing high-order syndromes, the registers of HOSUs for undecodable interleaves should be reset to zeros.

We now determine the least number of the shift registers in HOSU. Assuming one interleave is determined as decodable at the first stage and the last stage is required by the decoding process, the number of high-order syndromes needed to be computed during the whole decoding process is $2 \times (t_0 - t_4) = 32$. The degree of $\Lambda(x)$ is no more than $t_4 = 12$. It is easy to derive from (12) that the degree of $\hat{\Delta}(x)$ is no more than $t_4 - 1 = 11$. Thus (16) can be rewritten as

$$\begin{aligned} S_{25} &= \lambda_0^{-1}\hat{\delta}_0, \\ S_{26} &= \lambda_0^{-1}(S_{25}\lambda_1 + \hat{\delta}_1), \\ &\vdots \\ S_{36} &= \lambda_0^{-1}(S_{35}\lambda_1 + \cdots + S_{25}\lambda_{11} + \hat{\delta}_{11}), \\ S_{37} &= \lambda_0^{-1}(S_{36}\lambda_1 + \cdots + S_{26}\lambda_{11}), \\ &\vdots \\ S_{56} &= \lambda_0^{-1}(S_{55}\lambda_1 + \cdots + S_{44}\lambda_{12}). \end{aligned} \tag{25}$$

Only the last 12 syndromes need to be stored to compute each syndrome since the degree of $\Lambda(x)$ is no more than 12.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

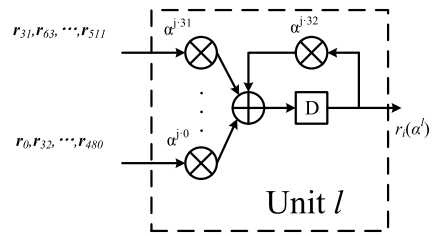


Fig. 9. The unit computing $r_i(\alpha^l)$ in 32-parallel in the symbol level.

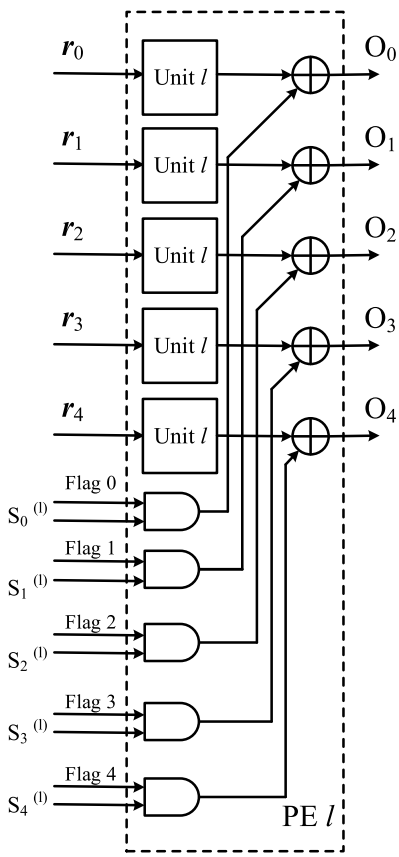


Fig. 10. The processing element computing $r_i(\alpha^l)$ and subtracting k_i from it.

Other cases can be analyzed similarly and in summary, it is sufficient to employ 16 registers for HOSU to store the syndromes. In addition, only $\lambda_0, \cdots, \lambda_{16}$ and $\hat{\delta}_0, \cdots, \hat{\delta}_{15}$ are involved in this unit.

The unit computing $r_i(\alpha^l)$ for interleave i is simply implemented as shown in Fig. 9. 32 symbols are read from the buffer and input to the unit in one clock cycle. After 16 clock cycles, $r_i(\alpha^l)$ is obtained. Fig. 10 shows the processing element (PE) computing $r_i(\alpha^l)$ and subtracting k_i from it for the syndrome of order l. If interleave i is determined as decodable after CS, flag i will be 1. For the decodable interleaves, their high-order syndromes $S_i^{(l)}$ will be subtracted from $r_i(\alpha^l)$.

The last step of computing the nested syndromes is matrix multiplication. The orders of the nested syndromes required by the first four stages are $25 \sim 26$, $27 \sim 32$, $33 \sim 40$, and $41 \sim 56$, respectively. For the decodable GII codeword, the maximum number of undecodable interleaves is $L-k-1$

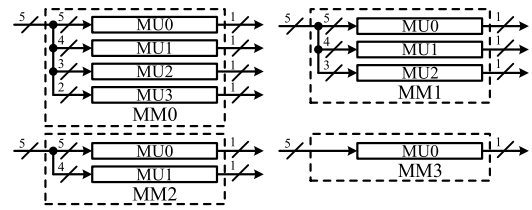


Fig. 11. The four types of matrix multipliers. Widths labeled on the nets in the figure denote the number of syndromes.

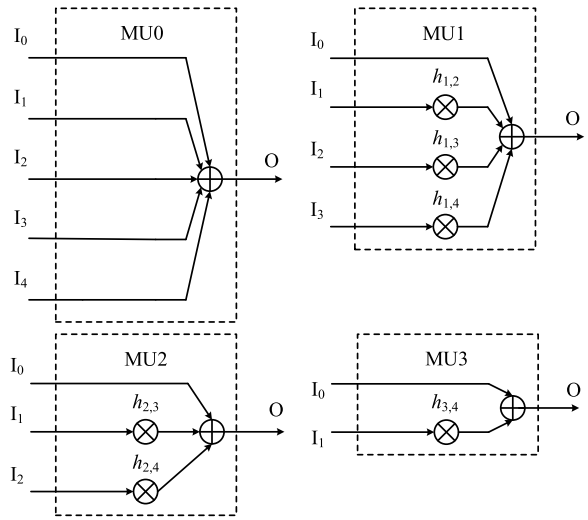


Fig. 12. The matrix multiplication units. MUi corresponds to the i-th row of $\boldsymbol{H}$.

after the k-th decoding stage for $0 \leq k < L$. Otherwise, a decoding failure will be declared. Thus the maximum number of the involved nested codes is also $L-k-1$ for the k-th stage. For the example code, they are 4, 3, 2, and 1, respectively. We design four types of matrix multipliers (MMs) as shown in Fig. 11. MM0, MM1, MM2, and MM3 can compute nested syndromes for at most 4, 3, 2, and 1 nested codes, respectively. Note that the widths labeled on the nets in the figure denote the numbers of syndromes instead of bit widths. The detailed architectures of matrix multiplication units (MUs) are shown in Fig. 12, where Mui corresponds to the i-th row of the conversion matrix $\boldsymbol{H}$. Since the maximum number of the required nested syndromes for one stage is 16, i.e., for the orders $41 \sim 56$, totally 16 MMs are employed in NSU.

Fig. 13 shows the architecture of NSU for the example code. 5 HOSUs are employed for all interleaves and 32 PEs are employed for the nested syndromes in 32 different orders. In interleave level, full parallelism is adopted. The net widths labeled on the outputs represent the maximum numbers of the output syndromes. For example, the first MM0 computes the order-25 nested syndromes for at most four nested codes at the first stage. However, if the high-order stages are required, it can also compute the order-27, order-33, and order-41 nested syndromes for at most 3, 2, and 1 nested codes, respectively. For the not used MMs, their inputs are set to zeros by the multiplexes.

The conversion matrix $\boldsymbol{H}$ utilized in our proposed decoder has a large number of identities and zeros, by which the

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

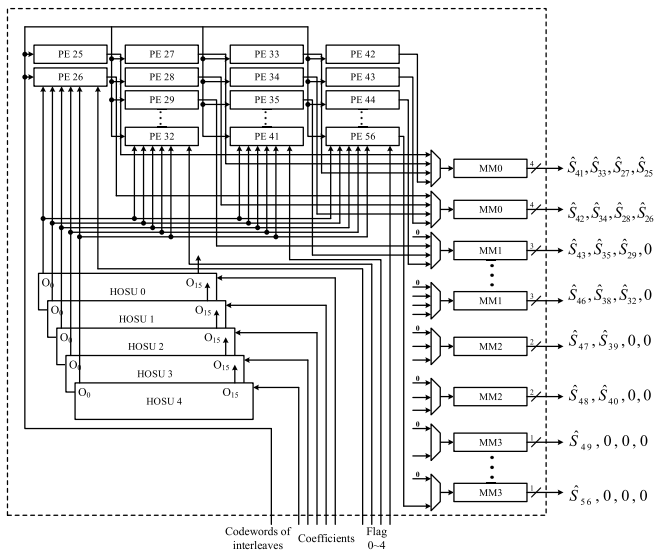


Fig. 13. The nested syndromes unit for the example code.

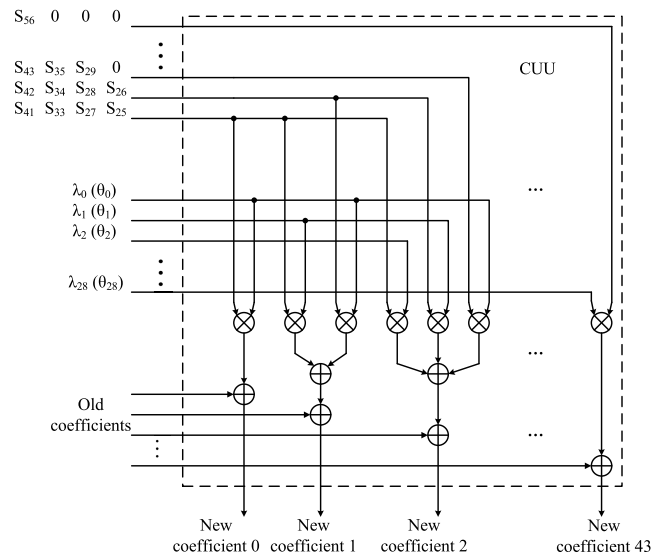


Fig. 14. The coefficients update unit for one interleave.

number of Galois field multipliers is reduced significantly. In our proposed NSU, the number of multipliers in all MMs is 38, while 54 multipliers will be required if the modified conversion matrix [6] is adopted. Thus over 29% multipliers in MMs are saved by the proposed decoder. If the original conversion matrix [5] is employed, the number of multipliers will be 64 and about 41% multipliers are saved by the proposed decoder. Moreover, the adders of MMs are also reduced by the utilized conversion matrix.

At the beginning of nested syndromes computation, one clock cycle is used to reset the registers of HOSUs to zeros for undecodable interleaves. Then high-order syndromes of all interleaves are computed in 16 clock cycles. Meanwhile, $r_i(\alpha^l)$ is computed by Unit l and CS is also performed at the same time. Then the high-order syndromes will be subtracted from $r_i(\alpha^l)$ by PE l according to the input flags. Finally, nested syndromes are obtained through the matrix multiplications. In summary, the number of clock cycles required by NSU is 18 in total. Note that, at the first decoding stage, another clock cycle is required by CS because of the method introduced to handle the miscorrection. Consequently, the number of clock cycles used to perform CS and computed the nested syndromes at the first stage is 19.

C. Coefficients Update Unit

For undecodable interleaves, previous results are reused to reinitialize the coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ before the new KES iterations. The reinitialization rules are given in (13) and Fig. 14 shows the coefficient update unit (CUU) for one interleave. Two such units are adopted to reinitialize the coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ for each interleave and totally ten units are adopted in the proposed decoder. It is essential to compare the architectures of HOSU and CUU. Although the computations of (19) and (16) are similar, both including a series of multiplications and additions, their architectures are designed with different degrees of parallelism in the proposed decoder. HOSU is designed to be fully serial in order to make full use of the clock cycles required by CS and reduce the hardware complexity. However, the updating of coefficients is expected to consume fewer extra clock cycles. Thus fully parallel architecture is exploited for CUU.

Since the largest error-correction capability is $t_0 = 28$, the considered coefficients of $\Lambda(x)$ are only $\lambda_0, \lambda_1, \cdots, \lambda_{28}$ in the proposed decoder. Considering the coefficients updating of the last stage, the first $2t_0 = 56$ coefficients of $\hat{\Delta}(x)$ ($\hat{\Theta}(x)$) are involved. We rewrite (13) as

$$\begin{aligned}
\hat{\delta}_0^{new} &= \underline{S_{41}\lambda_0} + \hat{\delta}_0^{old},\\
\hat{\delta}_1^{new} &= \underline{S_{42}\lambda_0 + S_{41}\lambda_1} + \hat{\delta}_1^{old},\\
&\vdots\\
\hat{\delta}_{43}^{new} &= \underline{S_{56}\lambda_{28}},\\
\hat{\delta}_{44}^{new} &= 0, \cdots, \hat{\delta}_{55}^{new} = 0.
\end{aligned} \tag{26}$$

Obviously, there is no need to reinitialize $\hat{\delta}_{44}, \cdots, \hat{\delta}_{55}$. For the reinitializations of other decoding stages, the numbers of coefficients need to be reinitialized are smaller and accordingly the number of CUU's outputs is 44.

D. Other Units and Discussion

LOSU computes the low-order syndromes of all interleaves in 32-parallel in symbol level and then uses them to initialize the KES in the first 16 clock cycles. The architecture of the unit computing one low-order syndrome $S_i^{(l)}$, for $1 \le l \le 24$ and $0 \le i < 5$, is very similar to the unit computing $r_i(\alpha^l)$ as shown in Fig. 9. The only difference is that the output is assigned to the input of the register in the low-order syndrome-computing unit.

CS is adopted to find out the roots of error locator polynomial $\Lambda(x)$ and then determine whether the interleave is decodable or not. An interleave is determined as decodable only when the degree of $\Lambda(x)$ is equal to the number of its roots. CS in the proposed decoder is also in 32-parallel in symbol level and totally 16 clock cycles are required to

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

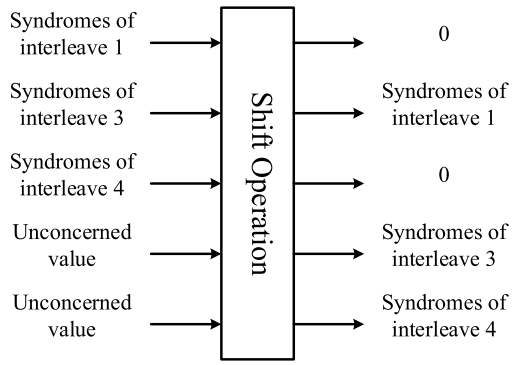


Fig. 15. The example shift operation. Interleave 1, interleave 3, and interleave 4 are all decodable and their high-order syndromes are reordered. Zeros are output for the undecodable interleaves.

complete the exhaustive search. In interleave level, five CS units are exploited for the five interleaves. One additional clock cycle is needed at the end of CS to check the equivalence of $\Lambda(x)$'s degree and the number of its roots. Then the flags are output to indicate whether the corresponding interleave is decodable or not. Moreover, at the first stage, the method introduced to handle the possible miscorrection needs one additional clock cycle. At the last stage, while CS is performed, error values for all interleaves are computed by Forney algorithm. For the interleaves determined as decodable with error-correction capability t_l, for $0 \leq l < L$, the error values can be computed through

$$Y_i = \frac{X_i^{-2t_l}\Omega^{(h)}(X_i^{-1})}{\Lambda'(X_i^{-1})} \qquad i = 1, \cdots, e, \tag{27}$$

which is obtaining from (20) through substituting t by t_l.

Nested syndromes are multiplied by $\boldsymbol{H}_{I^{un}}^{-1}$ to compute the high-order syndromes of undecodable interleaves as shown in (10). Instead of utilizing a general inverter, an LUT is employed in the inverse matrix (IM). The inputs of the LUT are the flags of all interleaves and the outputs are the entries of $\boldsymbol{H}_{I^{un}}^{-1}$. The multiplication between $\boldsymbol{H}_{I^{un}}^{-1}$ and nested syndromes is accomplished by a simple matrix multiplier.

Shift network is adopted to reorder high-order syndromes before coefficient updating. The inputs and outputs of the shift network are in nested code order and interleave order, respectively. Here we give an example shown in Fig. 15 to illustrate the shift operation. In this example, interleave 1, interleave 3, and interleave 4 are decodable and their high-order syndromes are arranged as the first three outputs of IM. After the shift operation, the high-order syndromes of decodable interleaves are reordered and zeros are added for the undecodable interleaves. The shifting is accomplished within the clock cycle of the polynomial reinitialization.

The codeword buffer is implemented by 10 dual-port static random access memories (DP-SRAMs), two of which are used to store one interleave. Since every 32 symbols are transmitted, $32 \times 9 = 288$ bits of one interleave are stored in its corresponding two DP-SRAMs in one clock cycle. The depth and width of one DP-SRAM are 16 and 144, respectively.

Except the iterative KES, other parts of the GII decoder are implemented or can be implemented as feed-forward architectures and pipelining can be applied to shorten the critical path at the cost of a few additional clock cycles, which are negligible when compared with the overall cycles required by the decoder. For instance, one-stage pipelining in CS architecture can only introduce L additional cycles for the L-layer GII decoder. For the 5-layer proposed GII decoder, five additional clock cycles are negligible when compared with the overall $169 - 15 = 154$ clock cycles. In last two subsections, we show that the HOSU can also be implemented as feed-forward. HOSU in the proposed decoder is designed in fully serial because the target throughput of the GII decoder can be achieved with such architecture. For decoders with higher speed, it can also be designed as a fully parallel architecture to be able to apply pipelining easily. However, pipelining is prohibitive in KES because of its inherent recursive architecture. As the largest error-correction capability of nested codes is t_0, $2t_0$ KES iterations are required by the decoding process in the worst cases. Obviously, even one-stage pipelining in KES can lead to $2t_0$ extra clock cycles. For the proposed decoder, $2t_0 = 56$ extra clock cycles are intolerable. Thus the critical path of KES forms the speed bottleneck of GII decoders. Although riBM-based decoder needs additional $L-1$ cycles to reinitialize the polynomials for an L-layer GII code, it is also negligible when compared with the overall clock cycles. Consequently, the proposed riBM-based GII decoder is more promising than the conventional BM-based decoder in ultrahigh-speed applications.

In this paper, our target throughput is set to 100 Gb/s in order to show the GII decoder's potentials in high-speed implementations. For higher throughput, higher degree of parallelism can be adopted. We constructed the 5-layer GII code only as an example to make it convenient to show the detailes of the decoder architecture. For different application scenarios, different GII codes may be constructed. The detailed architecture for a GII decoder heavily depends on the constraint of throughput, latency, and area, etc. In addition to the conventionally used SC, KES and CF, the proposed GII decoder has some other modules, such as NSU and CUU. It is easy to see the hardware complexity of CUU for one interleave is dominated by the error-correction capabilities. The complexity of NSU depends on the error-correction capabilities and the degrees of parallelism in both of the symbol and interleave levels. Besides, the latency of the GII decoder increases with the code length. Assuming full parallelism in the interleave level is adopted, the hardware complexity increases linearly with the number of layers.

V. Implementation Results

A. FPGA Implementation

The proposed decoder has been implemented on a Xilinx Virtex UltraScale XCVU440-flag2892 FPGA. Fig. 16 shows the error-correction performance comparison. Different from the simulation results presented in Section III, we performed the simulations in additive white Gaussian noise (AWGN) channel to highlight the net coding gain (NCG) of the proposed decoder. Besides, binary phase-shift

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

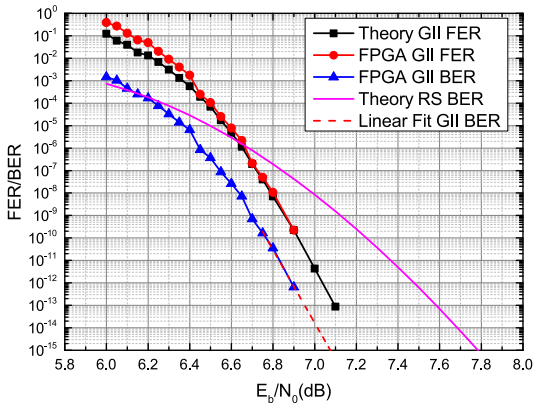


Fig. 16. The error performance curves drawn from FPGA simulation and computation results. The curve labeled as "Theory GII FER" is drawn from the results computed by (23) while the curves labeled as "FPGA GII FER" and "FPGA GII BER" represent the FER and BER curves drawn from the FPGA simulation results for the GII decoder. The "Linear Fit GII BER" is the linear fit curve deriving from the last three points of the "FPGA GII BER" curve. "Theory RS BER" curve is the performance curve for the simple interleaved RS codes.

keying (BPSK) modulation is considered. The curve labeled as "Theory GII FER" is drawn from the results computed by (23) while the curves labeled as "FPGA GII FER" and "FPGA GII BER" are both drawn from the FPGA simulation results. For each signal-to-ratio (SNR) point, the input BER of channel is obtained by statistics and then the theoretical FER is analyzed by (23).

The performance restriction of the code is that FER achieves 1×10^{-13} at BER of 1×10^{-3} and the code rate is $21393/22995 \approx 0.93$. Assuming the ratio between FER and BER is about 100, the output BER achieves 1×10^{-15} at input BER of 1×10^{-3}. In our simulations, it was observed that the ratio between FER and BER actually exceeds 100 and its value is $200 \sim 400$. Accordingly, it is extrapolated that the promising NCG of the constructed code is over 7.9 dB. The linear fitted curve of the last three points of the FPGA BER curve is drawn by a dashed line. Clearly, the output BER achieves 1×10^{-15} at the E_b/N_0 smaller than 7.1 dB and hence the NCG is over 7.9 dB. In the low-SNR region, the FPGA FER curve is slightly inferior to the Theory FER curve. We attribute the phenomenon to the inaccuracy of the double-precision floating-point computations of computers.

Our simulation step size is 0.1 dB between the last two simulation points. For the remaining simulated SNR points, the simulation step size is 0.05 dB. With E_b/N_0 smaller than 6.9 dB, no performance degradation induced by miscorrection is observed. However, at E_b/N_0 of 6.9 dB, slight performance degradation was observed. The method described at the end of Section III is adopted to eliminate the performance degradation. In our FPGA simulations without the method, the output FER and BER are 3.46×10^{-10} and 9.34×10^{-13} at E_b/N_0 of 6.9 dB. We evaluated all the incorrectly decoded codewords and found some of them are caused by the miscorrection of the first-layer RS codes while the others are exactly the undecodable codewords. For those miscorrected GII codewords, it is observed that only one interleave fails to be decoded correctly due to miscorrection at the first decoding stage. So the criterion is introduced to find such interleave and further decode it at the following stages. With the introduced method, we have output FER of 2.21×10^{-10} and BER of 6.47×10^{-13} for 6.9 dB. All the incorrectly decoded codewords had been evaluated and it was shown that the performance degradation caused by miscorrection is eliminated.

TABLE III
SYNTHESIS RESULTS OF THE PROPOSED DECODER

Characteristic	Result	Module	Area(mm^2)
Technology	28 nm	KES	0.12
Code Length	22995	CS	0.14
Code Rate	0.93	NSU	0.17
Supply Voltage	0.81 V	CUU	0.53
Frequency	833 MHz	Forney	0.30
Latency	184.8 ns	Others	0.19
Throughput	124 Gbps	-	-
Area	1.45 mm^2	-	-

TABLE IV
POWER DISSIPATION OF THE PROPOSED DECODER

E_b/N_0 (dB)	Average Power (mW)	Peak Power (mW)
6.0	147.4	702.6
7.1	137.0	418.6

Compared with the simple interleaved RS codes, GII codes can achieve better performance with the same code rate, due to the shared parities provided by the nested coding scheme. Here we consider the simple interleaved 5-layer RS code, each interleave of which belongs to the (511,475) RS code over GF(2^9). The simple interleaved RS code and the example GII code have almost the same code rate. As depicted in Fig. 16, the GII code can achieve an additional 0.7 dB coding gain at the output BER of 1×10^{-15}.

B. Synthesis Results and Power Dissipation

We also implement the proposed decoder using TSMC 28-nm CMOS technology. Table III lists the synthesis results and the areas of some key modules are also shown. As shown in Fig. 7, the average number of clock cycles for decoding one codeword is $C = 169 - 15 = 154$. The code length N is 22995 and the clock frequency is $f = 833$ MHz. Thus the throughput of the proposed decoder can achieve

$$Throughput = \frac{N}{C \cdot T} = \frac{N}{C \cdot 1/f} = 124 \text{ Gb/s}. \quad (28)$$

The riBM-based decoder for the simple interleaved RS code is also implemented. To achieve the same throughput as the proposed GII decoder, where one codeword is processed in 154 cycles on average, an 8-parallel architecture is adopted for each (511,475) RS subdecoder. Accordingly, the number of overall cycles for one codeword is $64 + 36 + 64 = 164$. Two codewords are designed to be overlapped in $164 - 154 = 10$ cycles. Using the same technology, the synthesis results at 883 MHz show that the area of the simple interleaved RS decoder including the codeword buffer is 0.28 mm^2. Thus it can be seen that the GII code achieves higher coding gain than the simple interleaved RS code at the cost of several times higher hardware complexity.

Moreover, Prime Time PX (PTPX) is used to analyze the power dissipation of the proposed decoder and the results are shown in Table IV. The power dissipations for E_b/N_0

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.

of 6.0 dB and 7.1 dB are analyzed and compared. For 7.1 dB, the codewords can be decoded correctly at the first decoding stage in almost all cases. However, at the E_b/N_0 of 6.0 dB, the average number of decoding stages is much larger, leading to higher power dissipation.

VI. Conclusion

An riBM-based GII decoding algorithm is first devised in this paper. Compared with the conventional BM-based decoding algorithm, the proposed one is more regular and promising in ultrahigh-speed implementations. To eliminate the performance degradation induced by miscorrection in the high-SNR region (with E_b/N_0 about 7.9 dB for the example code), a subtle method is introduced. The suspicious interleave is found by the criterion and forced to be decoded at the next decoding stage. Then a hardware decoder architecture for the constructed example code is developed. For the target high throughput, parallel processing techniques in coarse- and fine-grained levels are both adopted in the proposed decoder. We implemented the proposed decoder onto an FPGA and the FPGA simulation results show that the performance degradation induced by miscorrection is well eliminated by the introduced method. We also implement the proposed decoder using TSMC 28-nm technology and the synthesis results show the throughput can achieve 124 Gbps. Moreover, PTPX was used to analyze the power dissipation. At E_b/N_0 of 7.1 dB, where output BER achieves 1×10^{-15}, the average power consumption is 137.0 mW.

References

[1] T. Richardson, "Error-floors of LDPC codes," in *Proc. 41st Annu. Allerton Conf. Commun., Control, Comput.*, Oct. 2003, pp. 1426–1435.

[2] C. Di, D. Proietti, I. E. Telatar, T. J. Richardson, and R. L. Urbanke, "Finite-length analysis of low-density parity-check codes on the binary erasure channel," *IEEE Trans. Inf. Theory*, vol. 48, no. 6, pp. 1570–1579, Jun. 2002.

[3] M. A. Hassner, K. Abdel-Ghaffar, A. Patel, R. Koetter, and B. Trager, "Integrated interleaving—A novel ECC architecture," *IEEE Trans. Magn.*, vol. 37, no. 2, pp. 773–775, Mar. 2001.

[4] X. Tang and R. Koetter, "A novel method for combining algebraic decoding and iterative processing," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2006, pp. 474–478.

[5] Y. Wu, "Generalized integrated interleaved codes," *IEEE Trans. Inf. Theory*, vol. 63, no. 2, pp. 1102–1119, Feb. 2017.

[6] X. Zhang, "Modified generalized integrated interleaved codes for local erasure recovery," *IEEE Commun. Lett.*, vol. 21, no. 6, pp. 1241–1244, Jun. 2017.

[7] W. Li, J. Lin, and Z. Wang, "Comparison between generalized integrated interleaved codes and generalized error location codes," in *Proc. IEEE 18th Int. Conf. Commun. Technol. (ICCT)*, Oct. 2018, pp. 21–25.

[8] Y. Wu, "A new encoding method for integrated-interleaved codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2010, pp. 983–987.

[9] I. Zhilin and V. V. Zyablov, "LDPC code construction as a generalized concatenated code," in *Proc. 14th Int. Symp. Problems Redundancy Inf. Control Syst.*, 2015, pp. 107–110.

[10] M. Bossert, H. Grießer, J. Maucher, and V. V. Zyablov, "Some results on generalized concatenation of block codes," in *Applied Algebra, Algebraic Algorithms and Error-Correcting Codes*. Berlin, Germany: Springer, 1999, pp. 181–190.

[11] P. Huang, E. Yaakobi, and P. H. Siegel, "Multi-erasure locally recoverable codes over small fields," in *Proc. 55th Annu. Allerton Conf. Commun., Control, Comput.*, Oct. 2017, pp. 1123–1130.

[12] I. S. Reed and M. T. Shih, "VLSI design of inverse-free Berlekamp–Massey algorithm," *IEE Proc. E-Comput. Digit. Techn.*, vol. 138, no. 5, pp. 295–298, Sep. 1991.

[13] H. Burton, "Inversionless decoding of binary BCH codes," *IEEE Trans. Inf. Theory*, vol. IT-17, no. 4, pp. 464–466, Jul. 1971.

[14] D. V. Sarwate and N. R. Shanbhag, "High-speed architectures for Reed–Solomon decoders," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 9, no. 5, pp. 641–655, Oct. 2001.

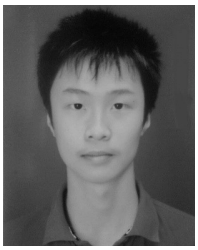
[15] X. Zhang, "Generalized three-layer integrated interleaved codes," *IEEE Commun. Lett.*, vol. 22, no. 3, pp. 442–445, Mar. 2018.

[16] M. Blaum and S. R. Hetzler, "Extended product and integrated interleaved codes," *IEEE Trans. Inf. Theory*, vol. 64, no. 3, pp. 1497–1513, Mar. 2018.

[17] R. E. Blahut, *Algebraic Codes for Data Transmission*. Cambridge, U.K.: Cambridge Univ. Press, 2003.

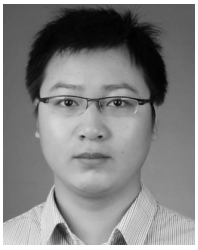
[18] Y. Wu, "New scalable decoder architectures for Reed–Solomon codes," *IEEE Trans. Commun.*, vol. 63, no. 8, pp. 2741–2761, Aug. 2015.

[19] T. K. Moon, *Error Correction Coding: Mathematical Methods and Algorithms*. Hoboken, NJ, USA: Wiley, 2005.



Wenjie Li received the B.E. degree in microelectronic science and engineering from Nanjing University, Nanjing, China, in 2017, where he is currently pursuing the M.S. degree in communication and information systems.

His current research interests include error correction codes and VLSI design for digital signal processing.



Jun Lin (S'14–M'16–SM'18) received the B.S. degree in physics and the M.S. degree in microelectronics from Nanjing University, Nanjing, China, in 2007 and 2010, respectively, and the Ph.D. degree in electrical engineering from Lehigh University, Bethlehem, in 2015.

From 2010 to 2011, he was an ASIC Design Engineer with AMD. In 2013, he was an Intern with Qualcomm Research, Bridgewater, NJ, USA. In 2015, he joined the School of Electronic Science and Engineering, Nanjing University, where he is currently an Associate Professor. His current research interests include low-power high-speed VLSI design, specifically VLSI design for digital signal processing and cryptography.

Dr. Lin was a member of the Design and Implementation of Signal Processing Systems (DISPS) Technical Committee of the IEEE Signal Processing Society. He was a co-recipient of the Merit Student Paper Award at the IEEE Asia Pacific Conference on Circuits and Systems in 2008. He was a recipient of the 2014 IEEE Circuits and Systems Society (CAS) Student Travel Award.

Zhongfeng Wang (M'00–SM'05–F'16) received the B.E. and M.S. degrees from Tsinghua University, Beijing, China, and the Ph.D. degree from the Department of Electrical and Computer Engineering, University of Minnesota, Minneapolis, MN, USA, in 2000.

He was with the National Semiconductor Corporation. From 2003 to 2007, he was an Assistant Professor with the School of EECS, Oregon State University, Corvallis, OR, USA. He joined Nanjing University as a Distinguished Professor in 2016, after serving Broadcom Corporation as a Leading VLSI Architect for nearly nine years. He is a world-recognized expert on VLSI for Forward Error Correction Codes. He has published approximately 200 papers, edited one book, and held tens of U.S. patents. During his tenure at Broadcom, he has contributed significantly on 10 Gb/s and beyond networking products. Additionally, he has made critical contributions in designing FEC coding schemes for 100- and 400-Gb/s Ethernet standards. His technical proposals have been adopted by many international networking standards. His current research interests include digital communications, machine learning, and efficient VLSI Implementation.

Dr. Wang has served as a technical program committee member (or Co-Chair), the session (or track) Chair, and a review committee member for tens of international conferences. He was a recipient of the IEEE Circuits and Systems Society VLSI Transactions Best Paper Award in 2007. In 2013, he served with the Best Paper Award Selection Committee for the IEEE Circuits and System Society. Since 2004, he has been serving as an Associate Editor of the IEEE Transactions on Circuits and Systems-I (TCAS-I), TCAS-II, and the IEEE Transactions on VLSI Systems, for numerous terms.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on April 04,2026 at 16:48:20 UTC from IEEE Xplore. Restrictions apply.